have a shopping cart, a dynamically populated menu, and a login widget all on one page. Each of those sections is independent of the main page content and has its own logic and data-access needs. In an ASP.NET Core app using Razor Pages, you'd implement each of those as a view component.

In section 32.3 I'll show you how to create a custom validation attribute. As you saw in chapter 6, validation is a key responsibility of Razor Page handlers and action methods, and the `DataAnnotations` attributes provide a clean, declarative way of doing so. We previously looked only at the built-in attributes, but you'll often find you need to add attributes tailored to your app's domain. In section 32.3 you'll see how to create a simple validation attribute and how to extend it to use services registered with the DI container.

Throughout this book I've mentioned that you can easily swap out core parts of the ASP.NET Core framework if you wish. In section 32.4 you'll do that by replacing the built-in attribute-based validation framework with a popular alternative, FluentValidation. This open-source library allows you to separate your binding models from the validation rules, which makes building certain validation logic easier. Many people prefer this approach of separating concerns to the declarative approach of `DataAnnotations`.

When you're building pages with Razor Pages, one of the best productivity features is Tag Helpers, and in the next section you'll see how you can create your own.

32.1 Creating a custom Razor Tag Helper

In this section you'll learn how to create your own Tag Helpers, which allow you to customize your HTML output. You'll learn how to create Tag Helpers